Polytechnic
University
of Coimbra

Python Language Programming

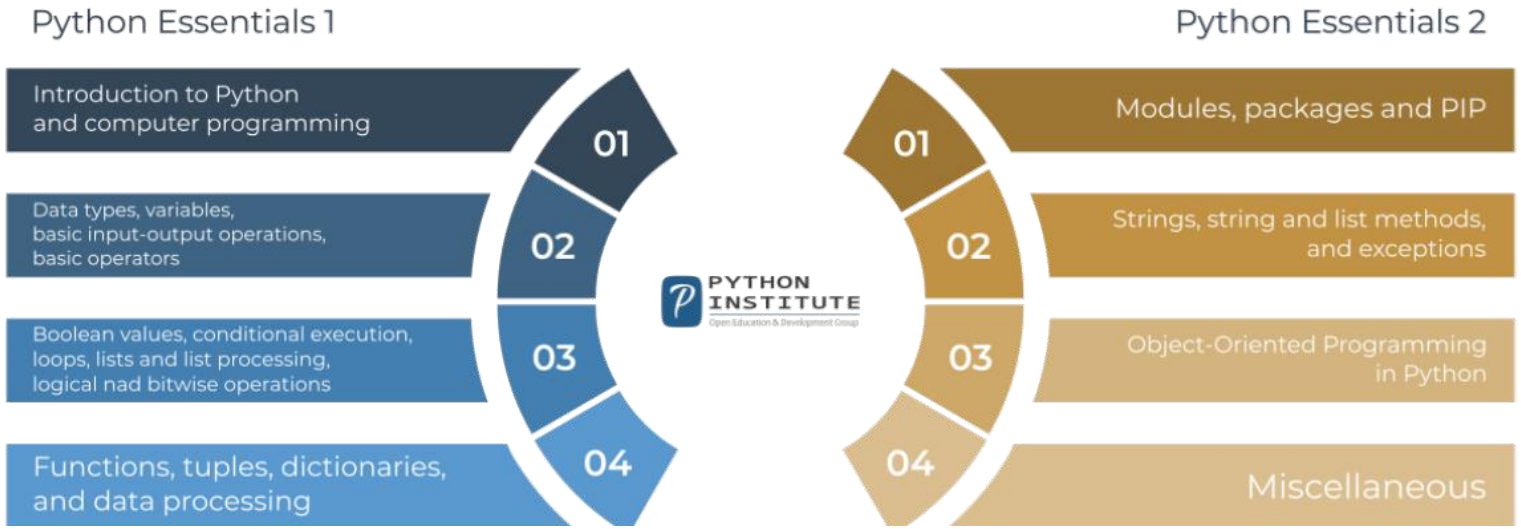
Functions

Polytechnic University of Coimbra

Python Language Programming

Python INSTITUTE
Open Education & Development Group

- **Disclaimer**
  - These slides and notes are based on the contents of the courses:
    - **Python Essentials 1 and 2**
      - aligned with the PCEP and PCAP certifications, respectively;
      - developed by the OpenEDG Python Institute; (www.pythoninstitute.org)
      - offered in the Cisco Networking Academy program.

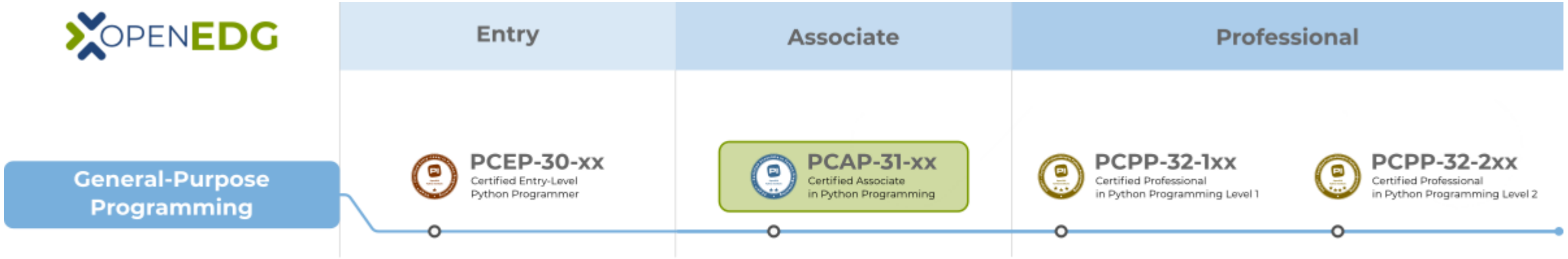


Polytechnic University of Coimbra

Python Language Programming

Python INSTITUTE
Open Education & Development Group

- **Disclaimer**
  - These slides and notes are based on the contents of the courses:
    - **Python Essentials 1 and 2**
      - aligned with the PCEP and PCAP certifications, respectively;
      - developed by the OpenEDG Python Institute; (www.pythoninstitute.org)
      - offered in the Cisco Networking Academy program.



OpenEDG Python Institute Certification Roadmap for the General-Purpose Programming track

Python INSTITUTE

Polytechnic University of Coimbra

Table of Contents

Polytechnic University of Coimbra

Python Language Programming

- **Functions**
  - can receive zero or more parameters, and generate zero or more outputs
  - comparison between Python and C functions:

C	Python
• can return at most one result • necessary to declare the type of both parameters and result • if necessary, it is possible to define the function prototype	• can return more results • NOT necessary to declare the type of either parameters or result • defined/redefined dynamically: NO prototypes

```c
int myFunc(int a, int b){
    printf("Sum: %d", a + b);
    return 0;
}
myFunc(4, 9);
```

Sum: 13

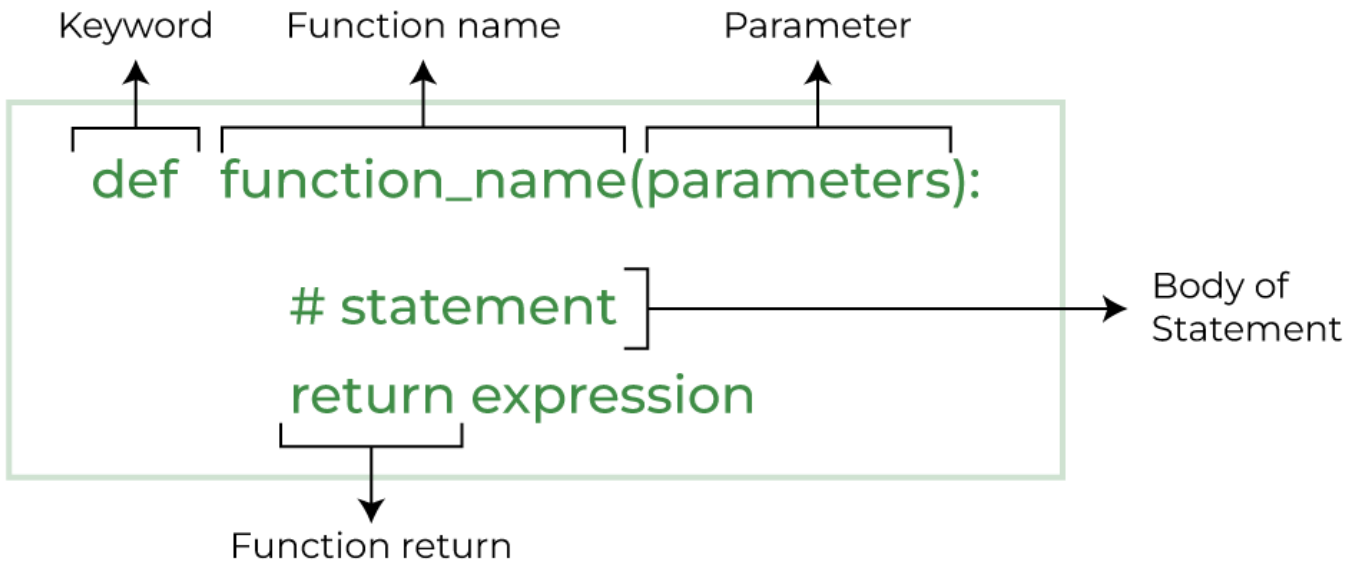
```python
def myFunc(a, b):
    print("Sum: " + str(a + b))
myFunc(4, 9)
```

Sum: 13

Polytechnic University of Coimbra

Python Language Programming

- **Functions**



Polytechnic University of Coimbra

Python Language Programming

- **Functions**
  - **Parameters**
    - exist only inside functions
    - defined in the space between the parentheses of the `def` statement;
    - assigning a value to the parameter is done at the time of the function's invocation, by specifying the corresponding argument.

```
def divide(num, denom):
    q = num // denom
    r = num % denom
    return q, r
```

Polytechnic University of Coimbra

Python Language Programming

- **Functions**
  - **Parameters**
    - arguments can be passed to a function using the following techniques:
      - **positional argument passing:** order of arguments matters;
      - **keyword (named) argument passing:** order doesn't matter;
      - **a mix of positional and keyword argument passing:** positional arguments must be put before keyword arguments.

```
def subtra(a, b):
    print(a - b)
subtra(5, 2)
subtra(2, 5)
```

```
3
-3
```

```
def subtra(a, b):
    print(a - b)
subtra(a=5, b=2)
subtra(b=2, a=5)
```

```
3
3
```

```
def subtra(a, b):
    print(a - b)
subtra(5, b=2)
subtra(5, 2)
```

```
3
3
```

Polytechnic University of Coimbra

Python Language Programming

- **Functions**
  - **Parameters**
    - **default (predefined) values:**

```
def introduction(first_name, last_name="Smith"):
    print("Hello, my name is", first_name, last_name)

introduction("James", "Doe")
introduction(first_name="William")
```

```
Hello, my name is James Doe
Hello, my name is William Smith
```

Polytechnic University of Coimbra

Python Language Programming

- **Functions**
  - **Lists and functions**

list as a function's argument

```
def hi_everybody(my_list):
    for name in my_list:
        print("Hi,", name)
hi_everybody(["Adam", "John", "Lucy"])
```

```
Hi, Adam
Hi, John
Hi, Lucy
```

list as a function's result

```
def create_list(n):
    my_list = []
    for i in range(n):
        my_list.append(i)
    return my_list
print(create_list(5))
```

```
[0, 1, 2, 3, 4]
```

Polytechnic University of Coimbra

Python Language Programming

- **Functions**
  - **Functions and scopes**
    - A variable existing outside a function has a scope inside the functions' bodies
      - excluding those of them which define a variable of the same name.

```
def my_function():
    print(var)


var = 1
my_function()
print(var)
```

1
1

```
def my_function():
    var = 2
    print(var)

var = 1
my_function()
print(var)
```

2
1

Polytechnic University of Coimbra

Python Language Programming

- **Functions**
  - **Functions and scopes**
    - **global keyword**
      - extends a variable's scope in a way which includes the functions' bodies

```
def my_function():
    global var
    var = 2
    print(var)

var = 1
my_function()
print(var)
```

2
2

Polytechnic University of Coimbra

Python Language Programming

- **Functions**
  - **Functions and scopes**

```
def test_scope():
    print(var)

var = 10
print(var)
test_scope()
print(var)
```

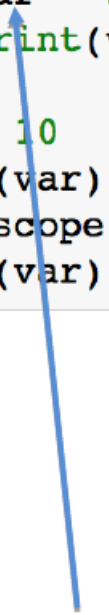
```
10
10
10
```

global **var**

```
def test_scope():
    var = 5
    print(var)

var = 10
print(var)
test_scope()
print(var)
```

```
10
5
10
```

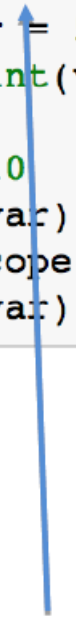


var redefined by **test_scope**

```
def test_scope():
    global var
    var = 5
    print(var)

var = 10
print(var)
test_scope()
print(var)
```

```
10
5
5
```



Through **global**, test_scope edits **var**

Polytechnic University of Coimbra

Python Language Programming

- **Functions**
  - **Recursion**
    - technique where a function invokes itself
    - necessary to consider conditions to stop the chain of recursive invocations!!!
      - if not the program may enter an infinite loop

```
def factorial_function(n):
    if n < 0:
        return None
    if n < 2:
        return 1
    return n * factorial_function(n - 1)
```

0! = 1
1! = 1
2! = 1 * 2
3! = 1 * 2 * 3
4! = 1 * 2 * 3 * 4
:
n! = 1 * 2 * 3 * 4 * … * n-1 * n

Polytechnic University of Coimbra

Python Language Programming

- **Exceptions**
  - exceptions can be "catched" and handled by using the ***try-except* block**.
  - If inside the ***try*** block:
    - all instructions are performed successfully:
      - execution jumps to the point after the last line of the ***except*** block;
    - anything goes wrong:
      - the execution immediately jumps out of the block into the first instruction located in the ***except*** block.

```
try:
     :
     :
except:
     :
     :
```

Polytechnic University of Coimbra

Python Language Programming

Casting

- **int():** takes one argument (e.g., a string: int(string)) and tries to convert it into an integer;

```
value = int(5.5)
print('Reciprocal of', value, 'is', 1/value)
```

➡ Reciprocal of 5 is 0.2

- **float():** takes one argument (e.g., a string: float(string)) and tries to convert it into a float.

```
value = int(float("5.5"))
print('Reciprocal of', value, 'is', 1/value)
```

➡ Reciprocal of 5 is 0.2

- **Exceptions**
  - most useful built-in exceptions:
    - `ZeroDivisionError, ValueError, TypeError, AttributeError, SyntaxError, KeyboardInterrupt`

```
while True:
    try:
        number = int(input("Enter an int number: "))
        print(5/number)
        break
    except ValueError:
        print("Wrong value.")
    except ZeroDivisionError:
        print("Sorry. I cannot divide by zero.")
    except:
        print("I don't know what to do...")
```

➡

```
Enter an int number: 6.8
Wrong value.
Enter an int number: string
Wrong value.
Enter an int number: ++++
Wrong value.
Enter an int number: 0
Sorry. I cannot divide by zero.
Enter an int number: 25
0.2
```

Polytechnic University of Coimbra

Python Language Programming

- **Exceptions**
  - the **`else`** block:
    - executed when no exception has been raised inside the *try*
    - located after the last *except* block

```
def reciprocal(n):
    try:
        n = 1 / n
    except ZeroDivisionError:
        print("Division failed")
        return None
    else:
        print("Everything went fine")
        return n

print(reciprocal(2))
print(reciprocal(0))
```

```
Everything went fine
0.5
Division failed
None
```

Polytechnic University of Coimbra

Python Language Programming

- **Exceptions**
  - the **`finally`** block:
    - always executed
    - must be the last branch of the code designed to handle exceptions

```
def reciprocal(n):
    try:
        n = 1 / n
    except ZeroDivisionError:
        print("Division failed")
        n = None
    else:
        print("Everything went fine")
    finally:
        print("It's time to say goodbye")
        return n
print(reciprocal(2))
print(reciprocal(0))
```

```
Everything went fine
It's time to say goodbye
0.5
Division failed
It's time to say goodbye
None
```

Polytechnic University of Coimbra

Bibliography

- Burkov, A. (2019). The hundred-page machine learning book. Andriy Burkov.
- **Cisco. (2022). Python Essentials. Retrieved September 4, 2024, from https://www.netacad.com/**
- Geron, A. (2019). Hands-on machine learning with scikit-learn, keras, and tensorflow. O'Reilly.
- Ekman, M. (2022). Learning Deep Learning. Addison-Wesley.
- Mckinney, W. (2017). Python for data analysis: Data wrangling with pandas, numpy, and ipython. O'Reilly.
- Raschka, S., & Mirjalili, V. (2019). Python machine learning: Machine learning and deep learning with python, scikit-learn, and tensorflow. Packt Publishing.
- Russel, S., & Norvig, P. (2018). Artificial intelligence: A modern approach. Pearson Education Limited.